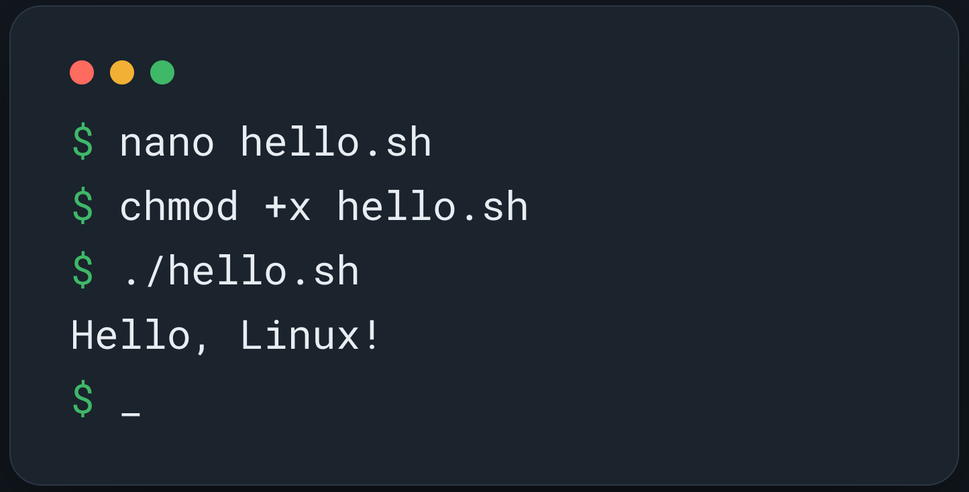
LINUX · УРОК 10

Bash: скрипты в командной оболочке

Alias, переменные, условия, строки, циклы, case, функции и массивы

Kaiyrkhan Mukapil



ПРОГРАММА КУРСА

Лекции

1 Дистрибутивы Linux, структура файловой системы, навигация (pwd, ls, cd)

2 Управление файлами и каталогами

3 Архивация и сжатие, мягкие и жёсткие ссылки

4 Текстовые редакторы и обработка текста

5 Пользователи и группы

6 Права доступа и владельцы файлов

7 Специальные права доступа

8 Пакеты и менеджеры пакетов

9 Устройства хранения данных

10 Сценарии оболочки Bash — сегодня

11 Сети в Linux

12 Межсетевой экран (firewall)

13 Файловый сервер Samba

14 Веб-сервер: Apache/Nginx, MySQL/MariaDB, PHP/Python

15 Повторение

ПРОГРАММА КУРСА

Лабораторные работы

1 Структура файловой системы Linux

2 Управление файлами и каталогами

3 Архивация и сжатие файлов

4 Работа с текстом

5 Пользователи и группы пользователей

6 Права доступа пользователей к файлам

7 Специальные права доступа

8 Управление пакетами

9 Устройства хранения данных

10 Скрипты в оболочке Bash — сегодня

11 Настройка сетевого интерфейса

12 Linux Firewall

13 Файловый сервер Samba

14 Веб-сервер и база данных в Linux

УРОК 10

План урока

01

Оболочка

Что такое shell, виды оболочек, alias

02

Первый скрипт

Shebang, chmod +x, способы запуска, комментарии

03

Данные и условия

Переменные, аргументы, read, арифметика, if

04

Строки и циклы

Подстроки, замена, for, while, until

05

case, функции, массивы

Ветвление по шаблону, local, return, массивы

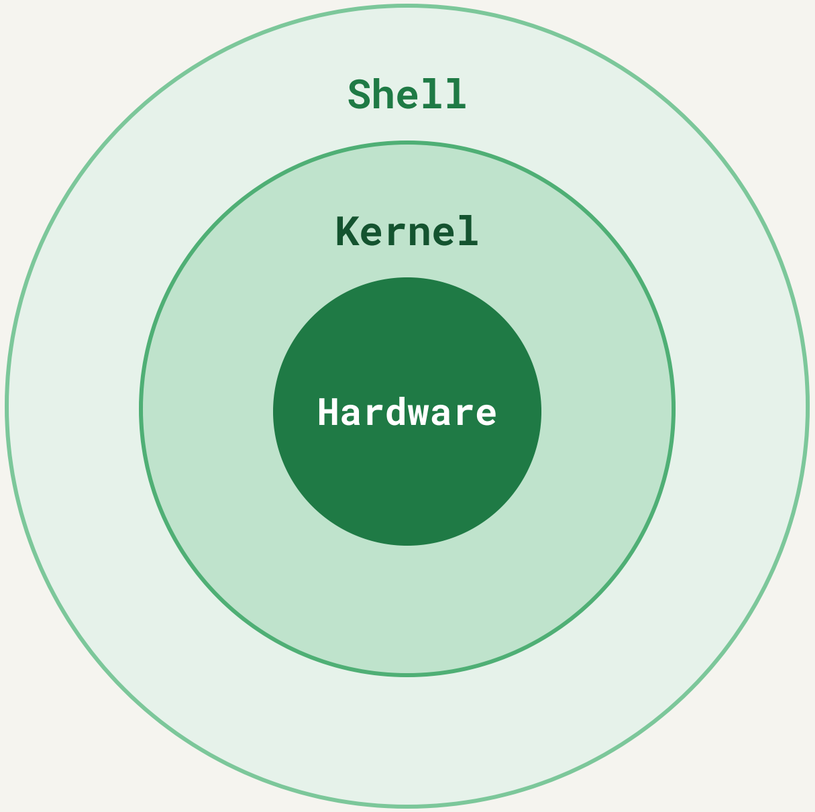
06

Лабораторная

60 минут в браузере: терминал, скрипты, автопроверка

ОБОЛОЧКА

Что такое оболочка (shell)



Shell — оболочка

Принимает команды пользователя и передаёт их ядру. Бывает CLI (bash в терминале) и GUI (графический рабочий стол).

Kernel — ядро

Сердце системы: управляет процессами, памятью, файлами и устройствами.

Hardware — аппаратура

Процессор, память, диски, сеть.

> **Bash** — The Bourne-Again Shell, самая распространённая оболочка Linux.

ОБОЛОЧКА

Два интерфейса: CLI и GUI

CLI · командная строка

bash, zsh, sh, fish

- команды вводятся текстом
- работает на серверах без графики и по SSH
- действия можно записать в скрипт и повторять

GUI · графический рабочий стол

GNOME, KDE Plasma, Xfce

- ещё: Cinnamon, MATE, LXQt (наследник LXDE)
- графика: X11 (X Window System) и Wayland — новый стандарт, по умолчанию в Ubuntu, Fedora, RHEL 10
- Display Manager — экран входа, Window Manager — управление окнами

ОБОЛОЧКА

Виды оболочек

Оболочка	Семейство	Где встречается
`sh`	Bourne	Исходная оболочка Unix; стандарт POSIX sh
`bash`	Bourne	По умолчанию в Ubuntu, Debian, Fedora, RHEL, Rocky
`dash`	Bourne	Быстрый /bin/sh в Debian и Ubuntu
`ksh`	Bourne	Korn shell: коммерческие Unix (AIX)
`zsh`	Bourne	По умолчанию в macOS, популярна у разработчиков
`csh, tcsh`	C shell	Синтаксис похож на язык C; исторически BSD
`fish`	Отдельно	Подсказки «из коробки», но не совместима с POSIX

Скрипт, написанный по стандарту POSIX, работает в любой оболочке семейства Bourne.

ОБОЛОЧКА

Как узнать свою оболочку

```
$ echo $SHELL       # оболочка входа
/bin/bash
$ echo $0           # что работает сейчас
bash
$ cat /etc/shells   # разрешённые оболочки
$ cat /etc/passwd | grep student
$ getent passwd student
student:x:1000:1000::/home/student:/bin/bash
$ chsh -s /bin/zsh  # сменить оболочку
```

$SHELL

Оболочка по умолчанию — последнее поле строки пользователя в /etc/passwd.

$0

Имя программы, которая выполняется сейчас. Внутри скрипта — имя самого скрипта.

ALIAS

Alias — короткое имя для команды

```
$ alias                 # список всех
$ alias c='clear'       # создать
$ alias ll='ls -l'
$ alias ping='ping -c4'
$ alias sshconfig='sudo vim /etc/ssh/sshd_config'
$ unalias c             # удалить
$ type ll
ll is aliased to `ls -l'
```

Чтобы алиас сохранился

```
$ echo "alias ll='ls -l'" >> ~/.bashrc
$ source ~/.bashrc
```

- без ~/.bashrc алиас пропадёт после закрытия терминала
- вокруг = не ставим пробелы
- \ls — запустить команду в обход алиаса
- в скриптах алиасы не работают — там пишут функции

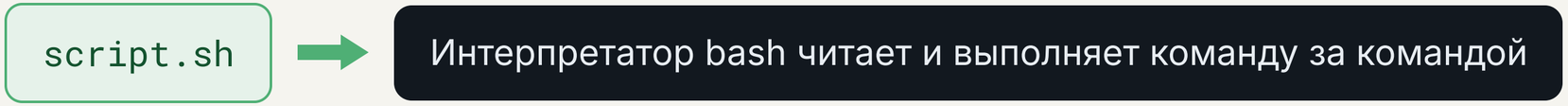
СКРИПТЫ

Как выполняется скрипт

Компилируемые языки: C, C++, Go



Интерпретируемые: Bash, Python



Java — смешанный путь: компилятор → байт-код → виртуальная машина JVM.

> Bash не проверяет скрипт целиком заранее: команды до ошибки успеют выполниться.

СКРИПТЫ

Первый скрипт за 4 шага

```
$ which -a bash
/usr/bin/bash
/bin/bash
$ nano script1.sh
#!/bin/bash
echo 'Hello World!'
$ chmod +x script1.sh  # или 755
$ ./script1.sh
Hello World!
```

Sharp

! Bang

#! **Shebang**

Shebang — первая строка: путь к интерпретатору, который выполнит файл. Переносимый вариант: #!/usr/bin/env bash

chmod +x даёт право на выполнение, **./** говорит «файл в текущей папке».

СКРИПТЫ

Способы запуска и типичные ошибки

Команда	Нужно право x	Что происходит
`./script.sh`	да	новый процесс, интерпретатор из shebang
`bash script.sh`	нет	новый процесс bash, shebang не важен
`source script.sh`	нет	в текущей оболочке: переменные и cd остаются
`bash -x script.sh`	нет	печатает каждую команду перед выполнением

`Permission denied`

Нет права x → chmod +x script.sh

`command not found`

Забыли ./ перед именем файла

`required file not found`

Ошибка в shebang, например #!bin/bash

COMMENT

Скрипт с комментариями

```
#!/bin/bash
mkdir -p dir1
# This is a Bash comment
echo 'Welcome to Almaty!' > dir1/file1.txt
ls -l dir1          # listing the directory
cat dir1/file1.txt # file contents
```

```
$ chmod +x script2.sh
$ ./script2.sh
```

Комментарий начинается с **#** и продолжается до конца строки — bash его пропускает.

Shebang тоже начинается с #, но это инструкция для системы, а не комментарий.

Несколько строк сразу:

```
: '
Это блок,
который bash не выполнит
'
```

VARIABLE

Переменные

```
name="Aigerim"          # без пробелов у =
age=19
echo "Привет, $name!" # Привет, Aigerim!
echo "Через год: $((age + 1))"
echo "${name}_backup"  # скобки отделяют имя
echo '$name'           # выведет $name
today=$(date +%d.%m.%Y)# вывод команды
readonly PI=3.14        # константа
unset name              # удалить
```

Частые ошибки

`name = "Ali"`

→ name: command not found

`echo $text`

без кавычек лишние пробелы схлопнутся — пишите "$text"

Имя: буквы, цифры и _, не с цифры. NAME и name — разные переменные.

VARIABLE

Аргументы и специальные переменные

Переменная	Значение
`$0`	имя скрипта
`$1 … $9, ${10}`	аргументы по номеру
`$#`	количество аргументов
`$@`	все аргументы по отдельности
`$?`	код последней команды (0 — успех)
`$$`	номер процесса (PID)
`$RANDOM`	случайное число 0–32767
`$HOME $USER $PWD`	переменные окружения

```
#!/bin/bash   # args.sh
echo "Скрипт: $0"
echo "Аргументов: $#"
echo "Все: $@"
echo "Первый: $1"
```

```
$ ./args.sh a b c
Скрипт: ./args.sh
Аргументов: 3
Все: a b c
Первый: a
```

VARIABLE

Ввод с клавиатуры и арифметика

```
read -p "Введите имя: " name
read -r a b           # два слова → две переменные
echo $(( a + b ))  $(( a * b ))
echo $(( 17 / 5 ))      # 3 — деление целое
echo $(( 17 % 5 ))      # 2 — остаток
echo $(( 2 ** 10 ))     # 1024 — степень
(( count++ ))          # увеличить на 1
let total=a+b
echo "scale=2; 10/3" | bc # 3.33
```

read ждёт строку со стандартного ввода и раскладывает слова по переменным.

Внутри **$(( ))** переменные пишут без $, пробелы можно ставить свободно.

Bash считает только **целые** числа. Для дробей — команда **bc**.

CONDITION

Условие if

```
#!/bin/bash
score=$1
if [ "$score" -ge 90 ]; then
    echo "Отлично"
elif [ "$score" -ge 50 ]; then
    echo "Сдал"
else
    echo "Не сдал"
    exit 1
fi
```

1. Внутри **[ ]** пробелы обязательны: `[ "$a" -gt 5 ]`
2. **[** — это команда test. Условие истинно, если команда вернула код 0.
3. Переменные в кавычках: пустая $a без кавычек даёт ошибку unary operator expected.
4. Каждый if закрывается fi, elif и else — по желанию.

CONDITION

Операторы сравнения

Числа

`-eq`	равно
`-ne`	не равно
`-lt`	меньше
`-le`	меньше или равно
`-gt`	больше
`-ge`	больше или равно

Строки

`=`	равны
`!=`	не равны
`-z`	пустая
`-n`	не пустая
`\< \>`	по алфавиту

Файлы

`-e`	существует
`-f`	обычный файл
`-d`	каталог
`-r -w -x`	права
`-s`	не пустой

```
[ -f data.txt ] && [ "$n" -gt 0 ]   # И
[ ! -d backup ] || echo "есть"      # НЕ, ИЛИ
```

CONDITION

[[ ]], (( )) и короткие условия

```
[[ $name == A* ]] && echo "Имя на A"              # шаблон
[[ $email =~ ^[a-z]+@[a-z]+\.kz$ ]] && echo OK # регулярное выражение
[[ $age -ge 18 && $age -le 25 ]]              # && внутри скобок
(( age >= 18 )) && echo "взрослый"           # арифметика
mkdir backup && cd backup                      # если успешно
cd /nope || echo "нет такой папки"         # если ошибка
```

[ ]

Классика POSIX, работает в любой оболочке.

[[ ]]

Только bash: шаблоны, =~, && и || внутри, меньше проблем с кавычками.

(( ))

Сравнение чисел привычными знаками < > == и без $.

STRING

Операции со строками

Запись	Результат	Что делает
`${#s}`	15	длина
`${s:0:6}`	report	подстрока
`${s: -3}`	txt	с конца
`${s%.*}`	report_2026	убрать хвост
`${s##*.}`	txt	убрать начало
`${s/2026/2027}`	report_2027.txt	заменить
`${s//r/R}`	RepoRt_2026.txt	заменить все
`${s^^}`	REPORT_2026.TXT	заглавные

```
s="report_2026.txt"
```

% отрезает с конца, # — с начала. Одинарный знак — кратчайшее совпадение, двойной — самое длинное.

,, — строчные, ^ — первая заглавная.

Склейка: full="$first $last"

LOOP

Цикл for

```
for i in 1 2 3; do echo $i; done
for i in {1..10}                # от 1 до 10
for i in {0..20..5}             # 0 5 10 15 20
for ((i = 0; i < 5; i++))       # как в языке C
for f in *.txt                 # все .txt в папке
for u in $(cut -d: -f1 /etc/passwd)

# таблица умножения
n=$1
for i in {1..10}; do
    echo "$n x $i = $((n * i))"
done
```

```
$ ./table.sh 7
7 x 1 = 7
7 x 2 = 14
…
7 x 10 = 70
```

for перебирает список: числа, слова, файлы или вывод команды. Тело цикла — между **do** и **done**.

LOOP

Циклы while и until

```
n=1
while [ $n -le 3 ]; do
    echo "Шаг $n"
    n=$((n + 1))   # без этого — бесконечно
done

until [ $n -eq 0 ]; do ((n--)); done

# чтение файла построчно
while read -r name score; do
    echo "$name: $score"
done < students.txt
```

while
повторяет, пока условие истинно

until
повторяет, пока условие ложно

break · continue
выйти из цикла · перейти к следующему шагу

CASE

Выбор по шаблону: case

```
read -p "Продолжить? (y/n) " answer

case "$answer" in
    y|Y|yes) echo "Продолжаем" ;;
    n|N|no)  echo "Отмена" ;;
    [0-9]*)  echo "Это число" ;;
    *)       echo "Не понял ответ" ;;
esac
```

Шаблоны — как у имён файлов: * любые символы, ? один символ, [abc] один из списка, | — «или».

;; завершает ветку, *) — «всё остальное».

Удобно для меню и команд вида start | stop | restart.

FUNCTION

Функции

```
greet() {
    local name=$1       # видна только внутри
    echo "Привет, $name!"
}
is_even() {
    (( $1 % 2 == 0 ))  # код 0 — «да»
}

greet "Дана"             # Привет, Дана!
if is_even 10; then echo "чётное"; fi
msg=$(greet "Тимур")    # вывод → переменная
```

Аргументы функции — тоже **$1, $2, $#, $@**.

return N возвращает код 0–255, а не значение. Значение отдают через **echo** и забирают через **$( )**.

local защищает переменные скрипта от случайной перезаписи.

Функцию объявляют выше места вызова.

ARRAY

Массивы

Обычный (индексы 0, 1, 2...)

```
colors=(red green blue)
echo ${colors[0]}     # red
echo ${colors[@]}     # все
echo ${#colors[@]}    # 3
colors+=(white)       # добавить
echo ${!colors[@]}    # 0 1 2 3
for c in "${colors[@]}"; do
    echo $c
done
```

Ассоциативный (ключ → значение)

```
declare -A age
age[ali]=20
age[dana]=19
echo ${age[dana]}     # 19
echo ${!age[@]}       # ключи
echo ${#age[@]}       # 2
```

Порядок ключей не гарантирован — для вывода используйте sort.

ПРАКТИКА

Отладка и хорошие привычки

`bash -x script.sh`

Печатает каждую команду с уже подставленными значениями. Внутри скрипта: set -x ... set +x.

`bash -n script.sh`

Только проверка синтаксиса, без выполнения.

`set -euo pipefail`

Остановиться при ошибке команды, при пустой переменной и при сбое в конвейере.

`shellcheck.net`

Онлайн-проверка скрипта: находит типичные ошибки и объясняет их. Устанавливать ничего не нужно.

Привычки: "$переменные" в кавычках · понятные имена · комментарии «зачем» · exit 0 при успехе и exit 1 при ошибке.

ПРАКТИКА

Всё вместе: отчёт по группе

```bash
#!/bin/bash
file=${1:-students.txt}
[ -f "$file" ] || { echo "Нет файла $file"; exit 1; }
grade() {
    local s=$1
    if (( s >= 90 )); then echo "A"
    elif (( s >= 75 )); then echo "B"
    elif (( s >= 50 )); then echo "C"
    else echo "F"; fi
}
declare -A count
while read -r last first score; do
    g=$(grade "$score")
    count[$g]=$(( ${count[$g]:-0} + 1 ))
    echo "$last $first $score $g"
done < "$file"
for g in A B C F; do
    echo "$g: ${count[$g]:-0}"
done
```

```
$ ./report.sh
Ахметов Али 85 B
Бекова Дана 42 F
Ким Тимур 67 C
Серикова Аружан 91 A
Жумабаев Ерлан 50 C
Оспанова Мадина 38 F
A: 1
B: 1
C: 2
F: 2
```

Здесь есть всё: значение по умолчанию ${1:-...}, проверка файла, функция с local, while read, ассоциативный массив и for.

ТАПСЫРМА · ЗАДАНИЕ

Шпаргалка: 9 тем урока

Alias

```
alias ll='ls -l'
```

Comment

```
# это комментарий
```

Variable

```
name="Ali"; echo "$name"
```

Condition

```
if [ "$a" -gt 5 ]; then … fi
```

String

```
${#s} ${s:0:3} ${s%.*}
```

Loop

```
for i in {1..5}; do … done
```

Case

```
case $x in a) … ;; *) … ;; esac
```

Function

```
f() { local x=$1; echo "$x"; }
```

Array

```
a=(x y z); ${a[@]} ${#a[@]}
```

ЛАБОРАТОРНАЯ РАБОТА №10

60 минут в браузере — без установки

1. Откройте файл **lab10_bash.html** в Chrome, Edge или Firefox. Внутри — терминал Linux, редактор и nano.
2. Введите фамилию, имя и группу.
3. Решайте по порядку: разминка → терминал → скрипты. Кнопка «Проверить» сразу запускает тесты.
4. В конце — «Скачать результат» и отправьте файл Lab10_Familiya_Imya_Gruppa.html преподавателю.

33

задания: 6 вопросов, 5 в терминале, 22 скрипта

100

баллов за основные задания

+46

бонус для тех, кто успеет больше

```
$ echo "Вопросы?"
```

Вопросы?

Дальше — лабораторная работа. Следующий урок: сети в Linux.